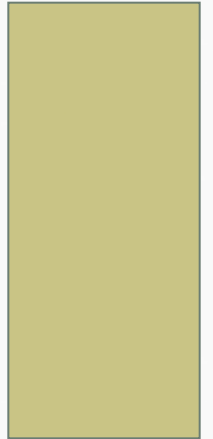


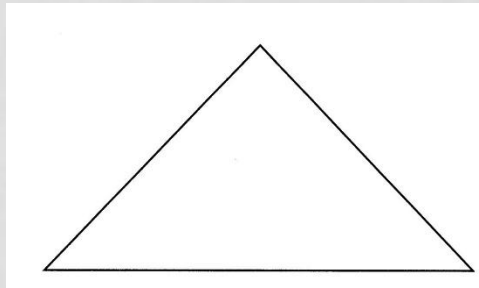
EC-200 DATA STRUCTURES

LECTURE 13 - HEAPS

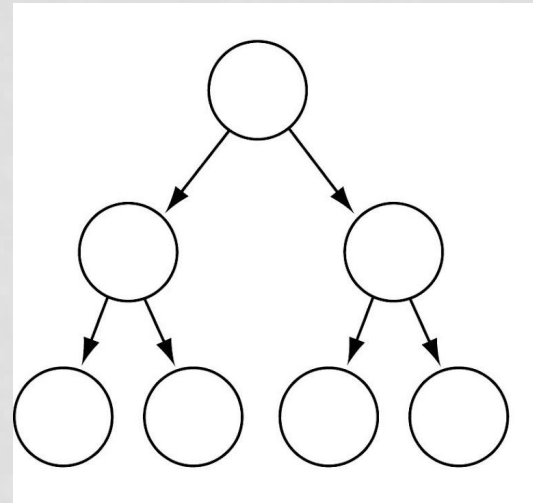


PERFECT BINARY TREE

- Every non-leaf node has two children
- All the leaves are on the same level

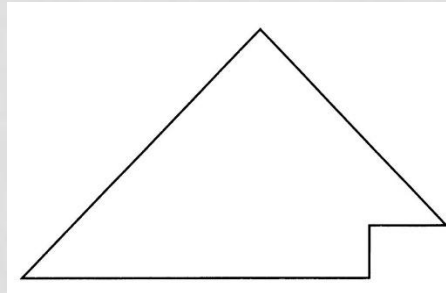


Perfect Binary Tree



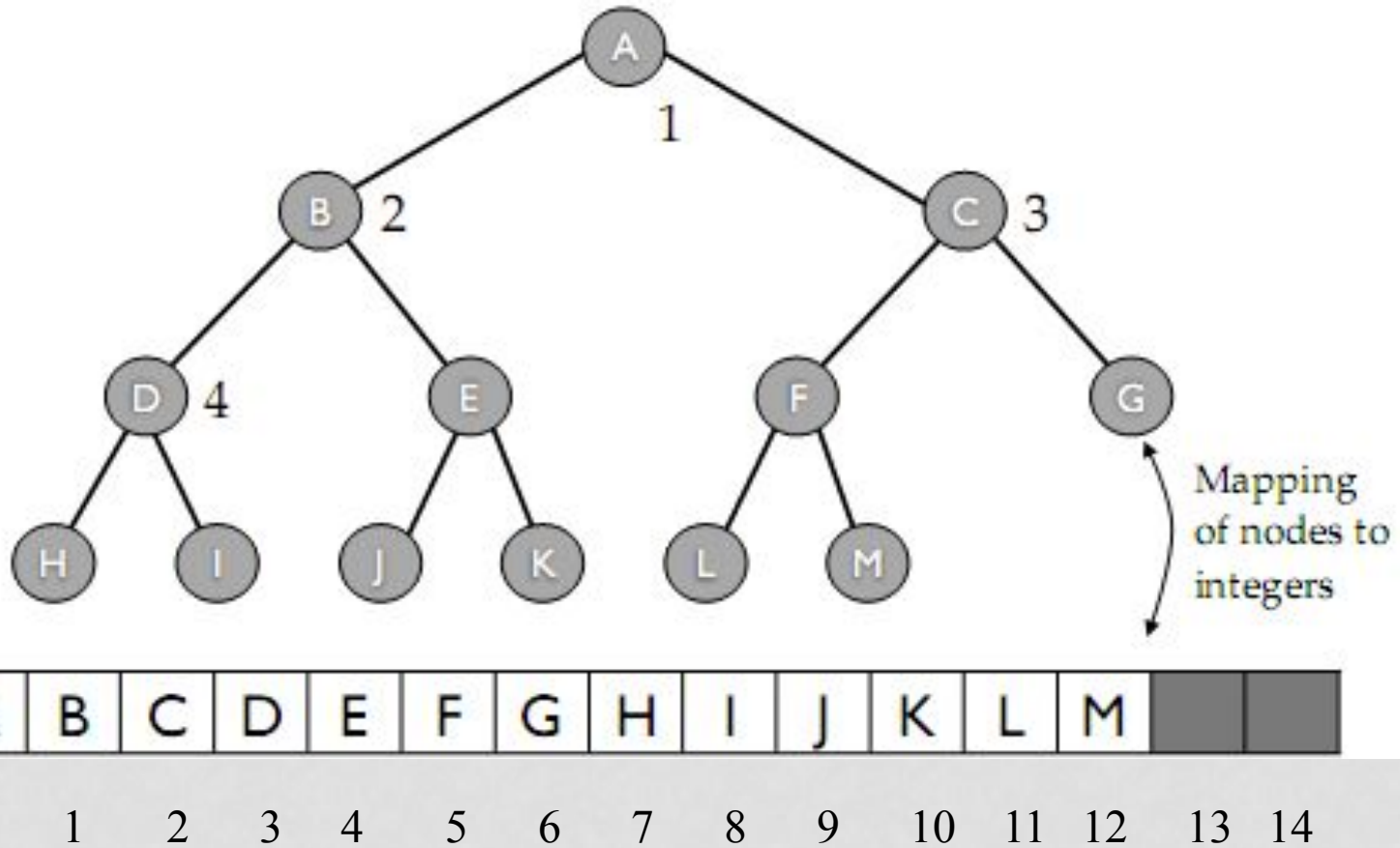
COMPLETE BINARY TREE

- A binary tree that is either perfect or perfect through the next-to-last level
- The last level is full from left to right (i.e., leaves are as far to the left as possible)



Complete Binary Tree

ARRAY IMPLEMENTATION FOR COMPLETE BINARY TREES



ARRAY-BASED REPRESENTATION OF BINARY TREES (CONT.)

- Parent-child relationships:

left child $nodes[index] = nodes[2*index+1]$

right child $nodes[index] = nodes[2*index+2]$

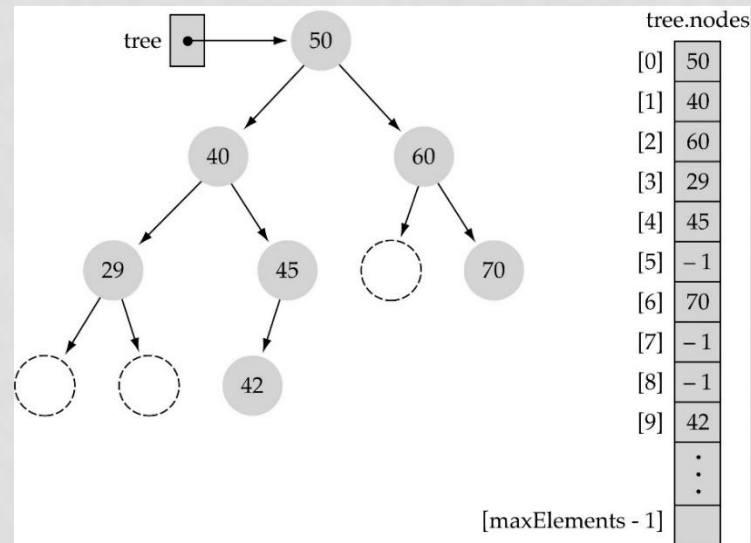
Parent $nodes[index] = tree.nodes[(index-1)/2]$
(**int** division-truncate)

- **Leaf nodes:**

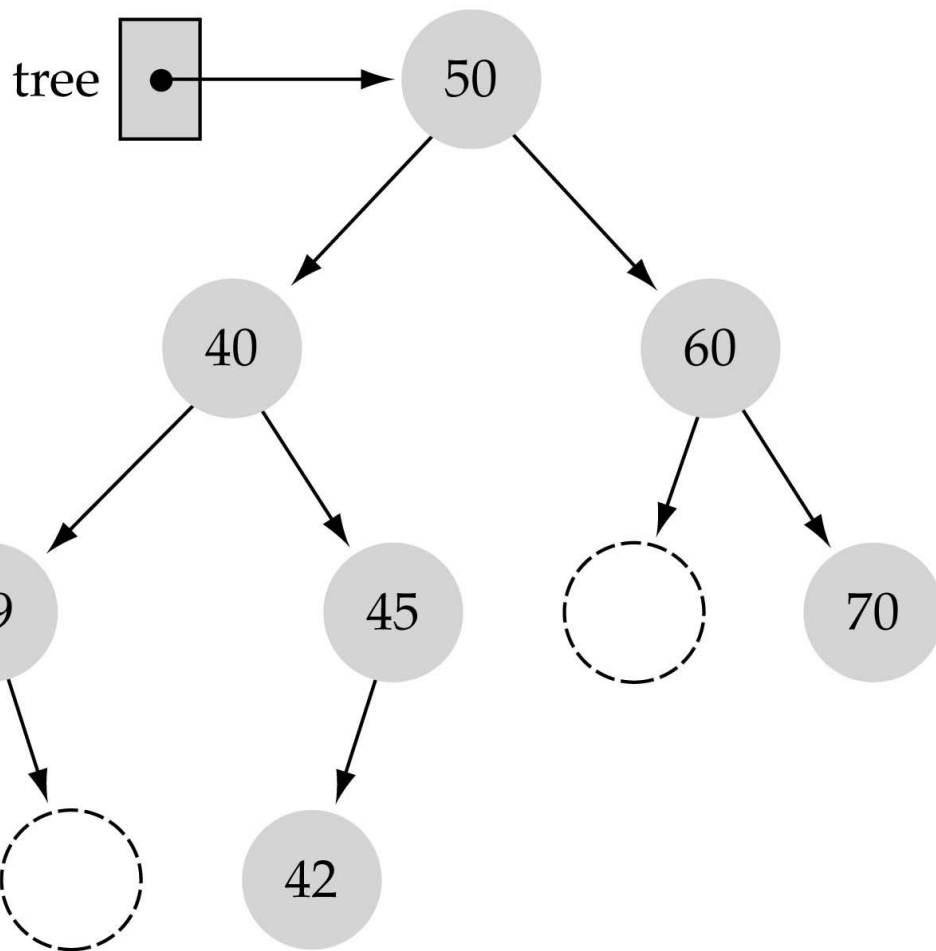
$nodes[numElements/2]$ to $nodes[numElements - 1]$
(**int** division-truncate)

ARRAY-BASED REPRESENTATION OF BINARY TREES (CONT.)

- Full or complete trees can be implemented easily using an array-based representation (elements occupy contiguous array slots)
- "Dummy nodes" are required for trees which are not full or complete



tree.numElements = 10



tree.nodes

[0]	50
[1]	40
[2]	60
[3]	29
[4]	45
[5]	- 1
[6]	70
[7]	- 1
[8]	- 1
[9]	42
	⋮
	⋮
	⋮

[maxElements - 1]

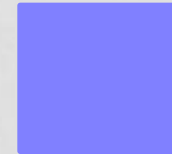
tree.numElements = 10

HEAPS

A heap is a certain kind of complete binary tree

HEAPS

Root



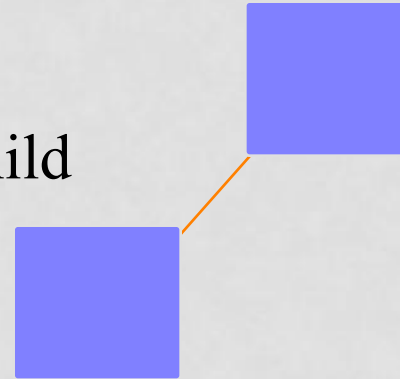
A heap is a certain kind of complete binary tree.

When a complete binary tree is built, its first node must be the root.

HEAPS

Complete
binary tree.

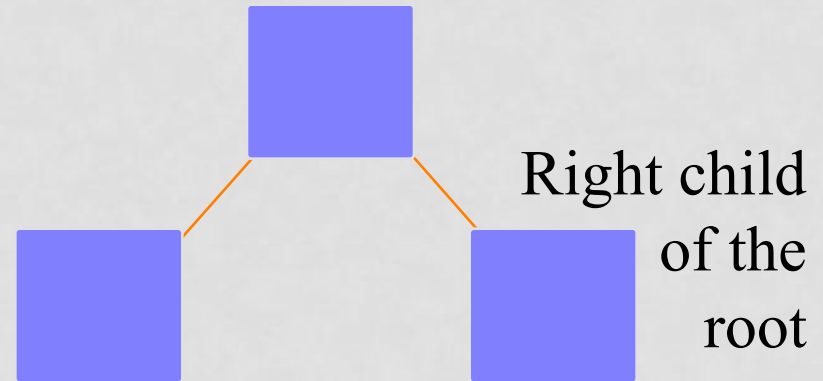
Left child
of the
root



The second node is
always the left child
of the root.

HEAPS

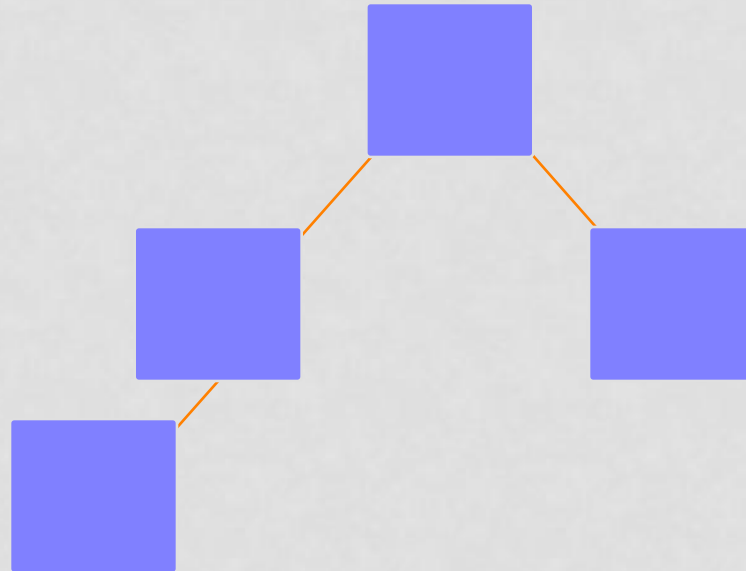
Complete
binary tree.



The third node is
always the right child
of the root.

HEAPS

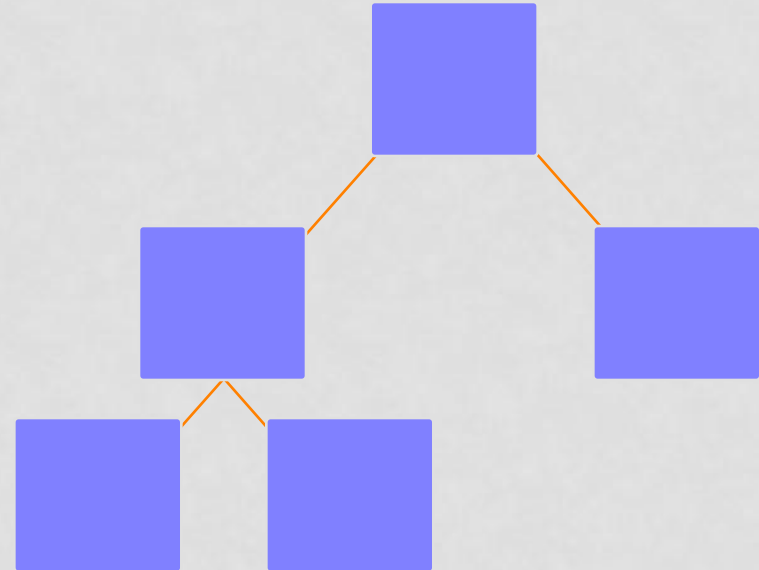
Complete
binary tree.



The next nodes
always fill the next
level from left-to-right.

HEAPS

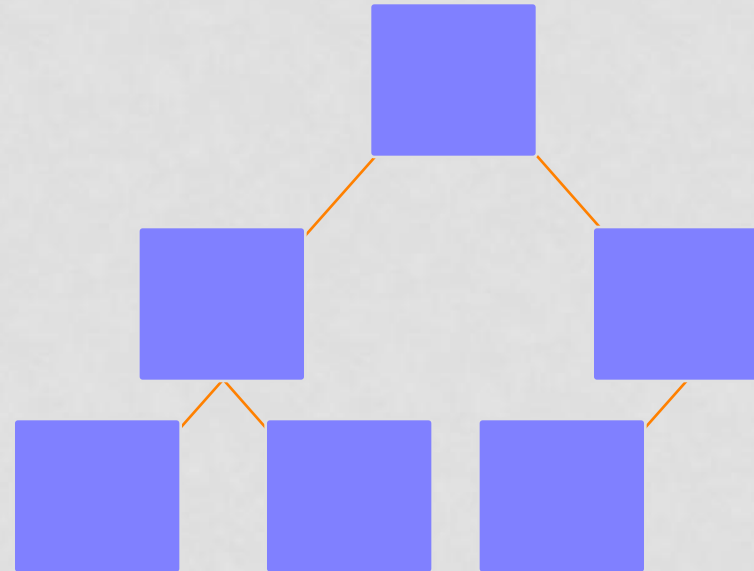
Complete
binary tree.



The next nodes
always fill the next
level from left-to-right.

HEAPS

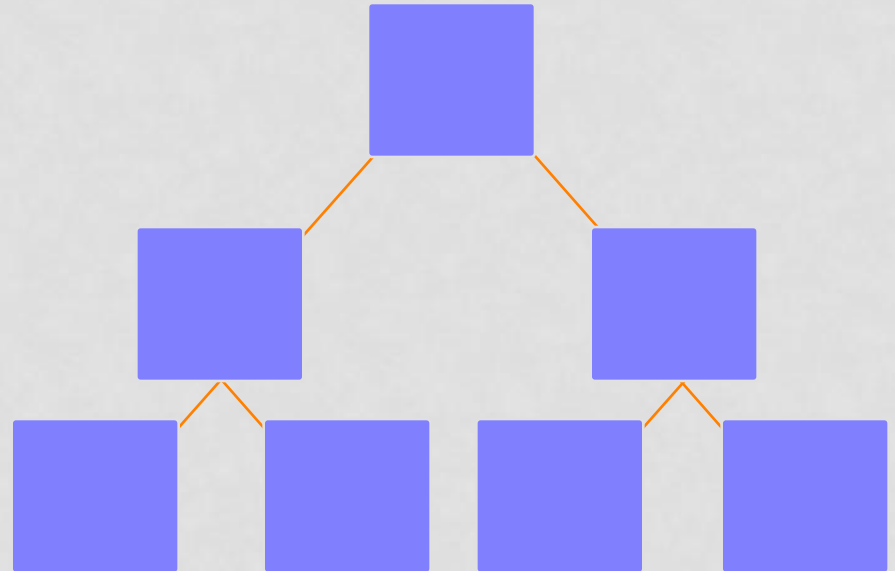
Complete
binary tree.



The next nodes
always fill the next
level from left-to-right.

HEAPS

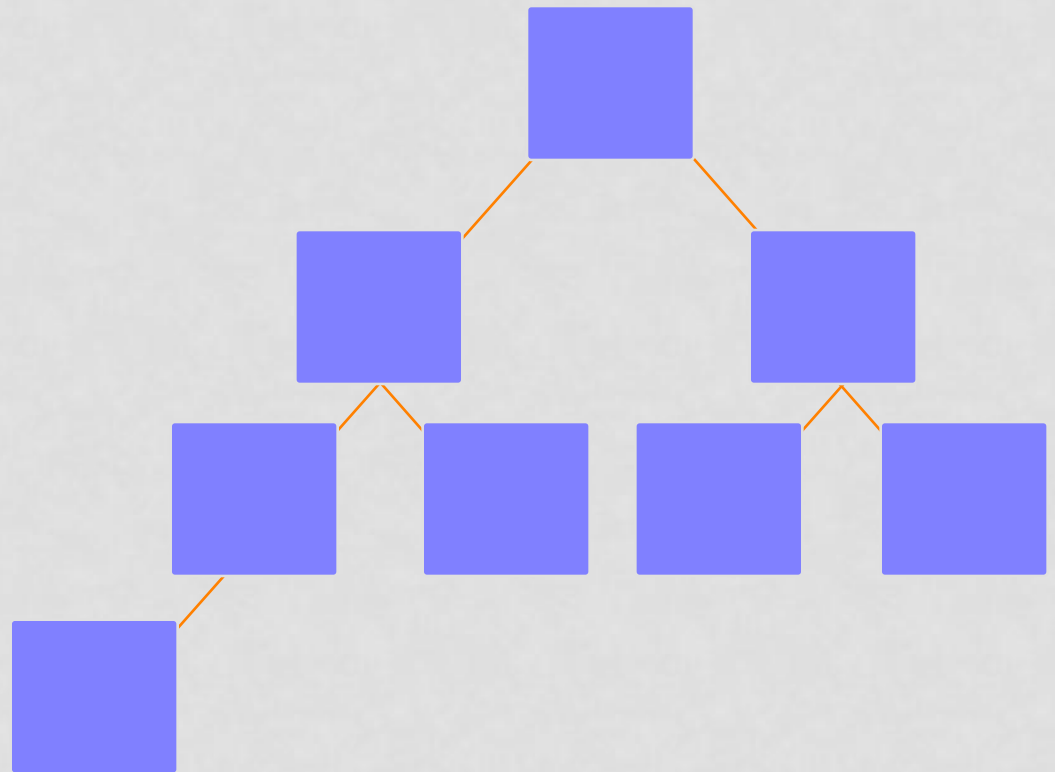
Complete
binary tree.



The next nodes
always fill the next
level from left-to-right.

HEAPS

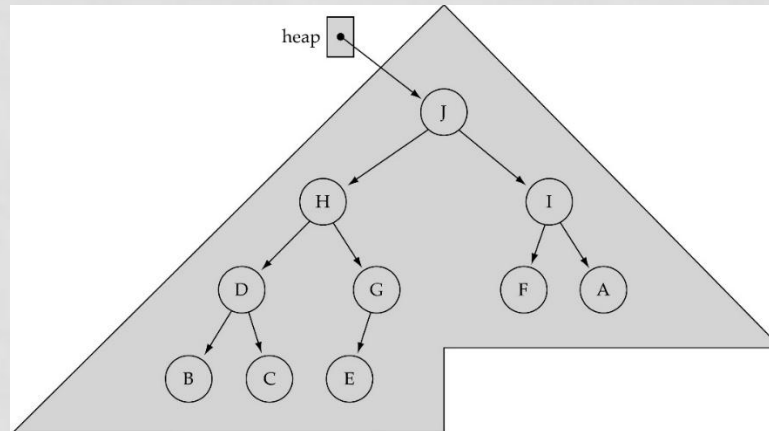
Complete
binary tree.



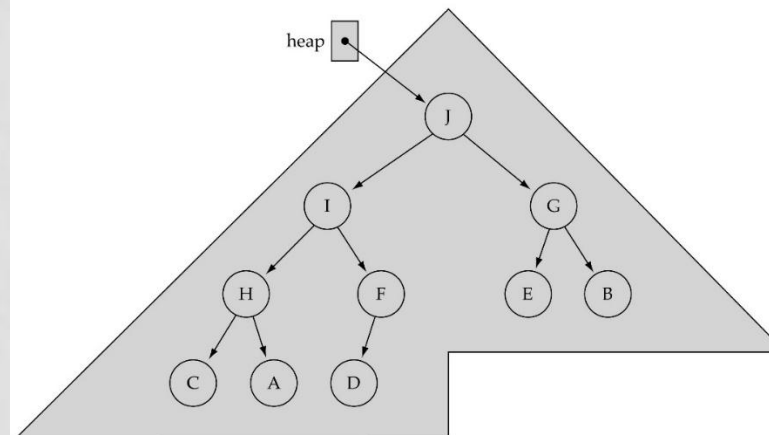
WHAT IS A HEAP?

- It is a binary tree with the following properties:
 - *Property 1*: it is a complete binary tree
 - *Property 2*: the value stored at a node is greater or equal to the values stored at the children (heap property)

WHAT IS A HEAP? (CONT.)



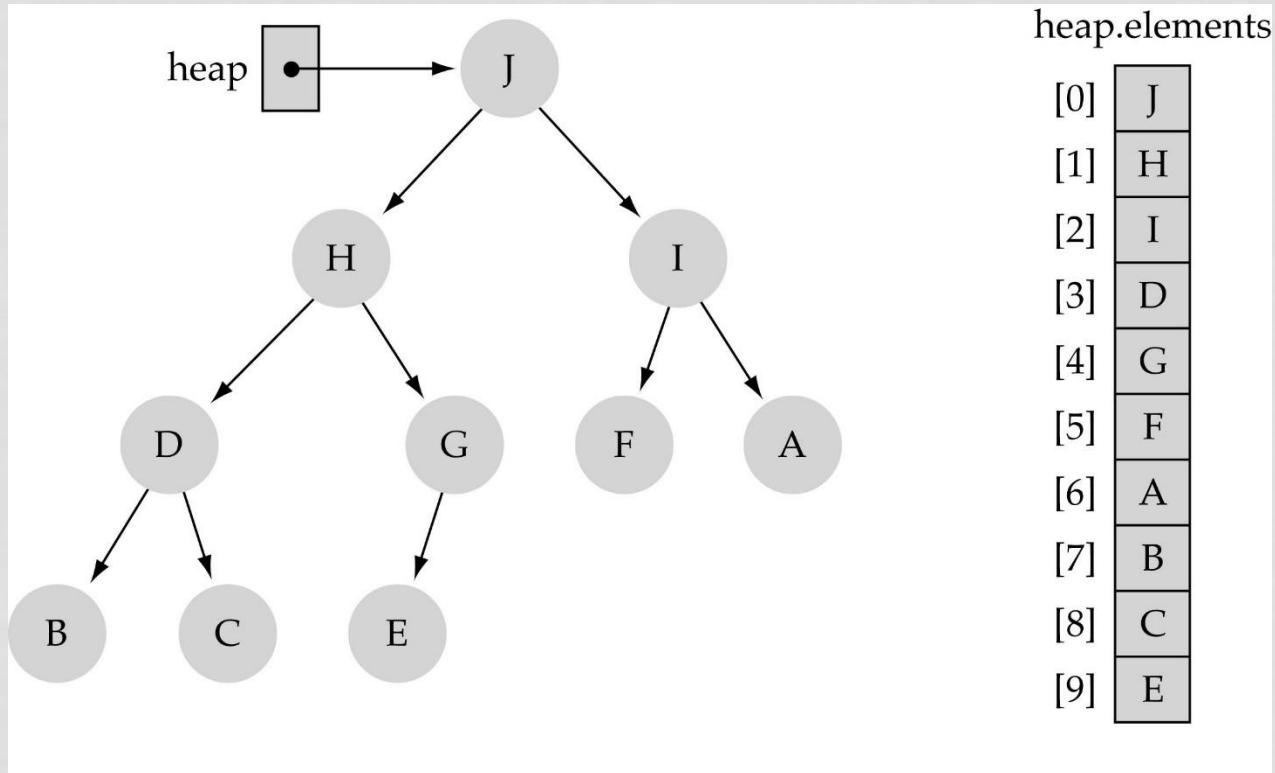
(a)



(b)

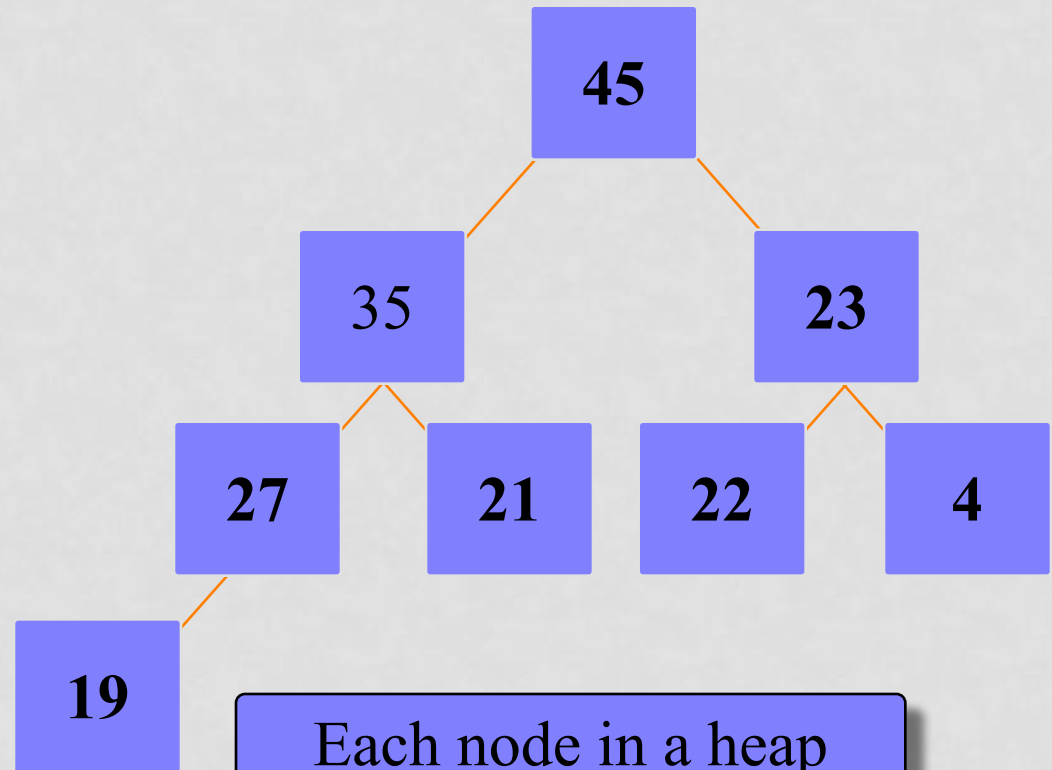
LARGEST HEAP ELEMENT

- From *Property 2*, the largest value of the heap is always stored at the root



HEAPS

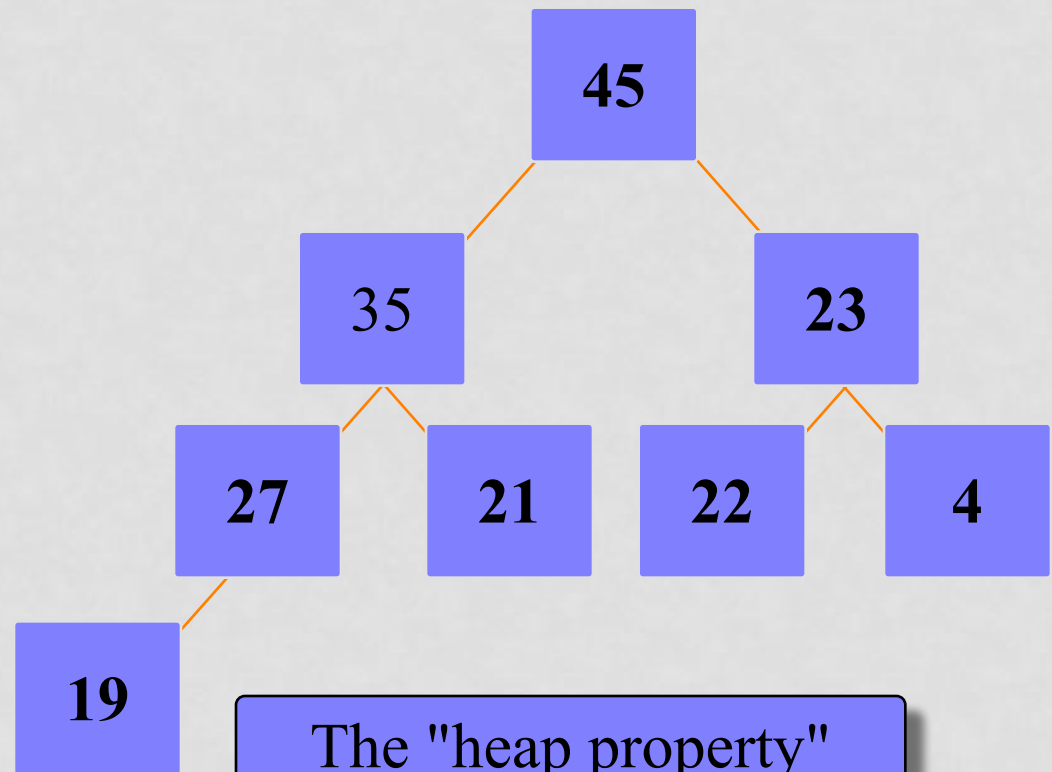
A heap is a certain kind of complete binary tree.



Each node in a heap contains a key that can be compared to other nodes' keys.

HEAPS

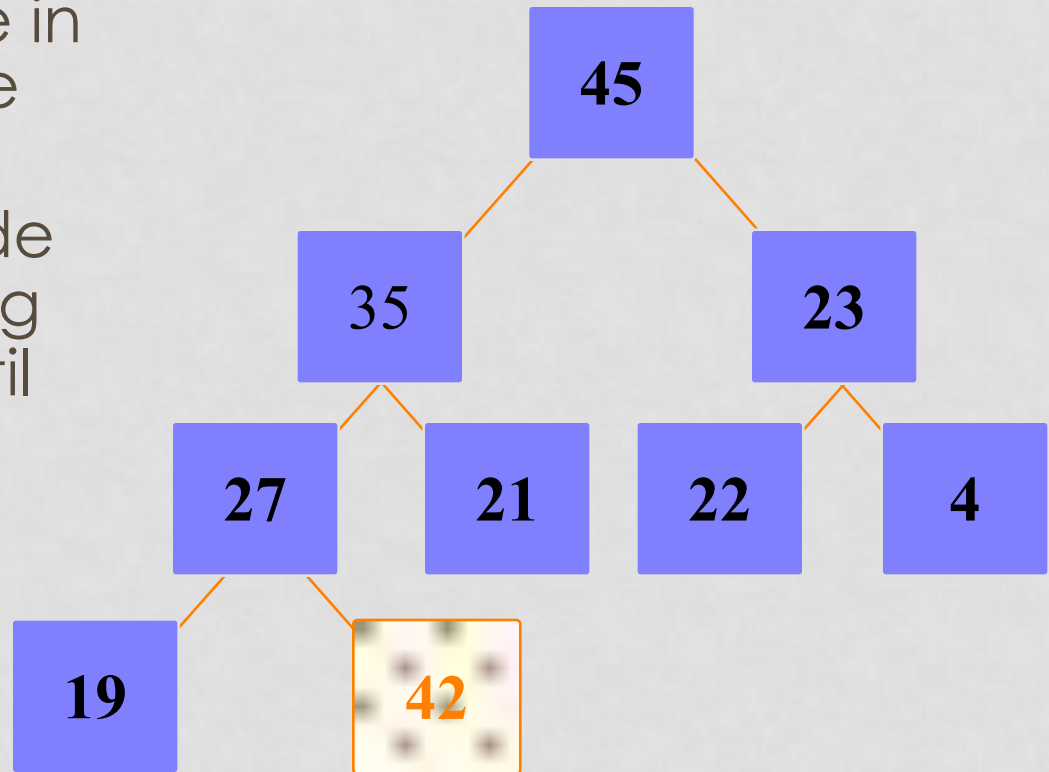
A heap is a certain kind of complete binary tree.



The "heap property" requires that each node's key is $\geq$ the keys of its children

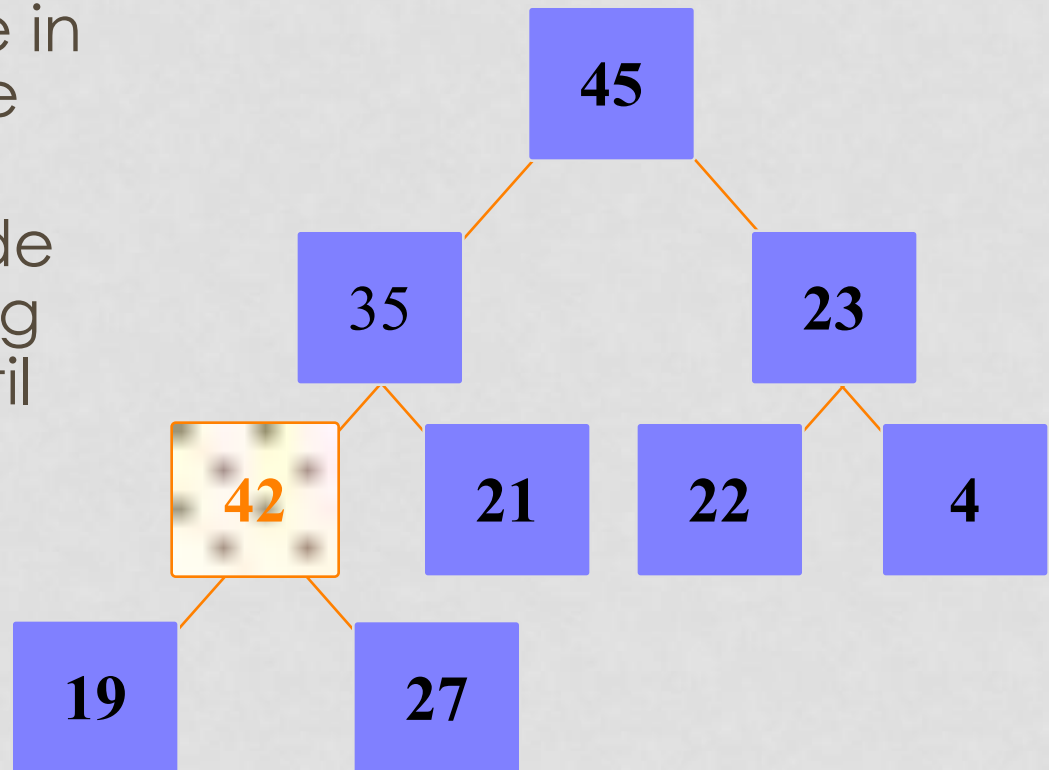
ADDING A NODE TO A HEAP

- ❑ Put the new node in the next available spot.
- ❑ Push the new node upward, swapping with its parent until the new node reaches an acceptable location.



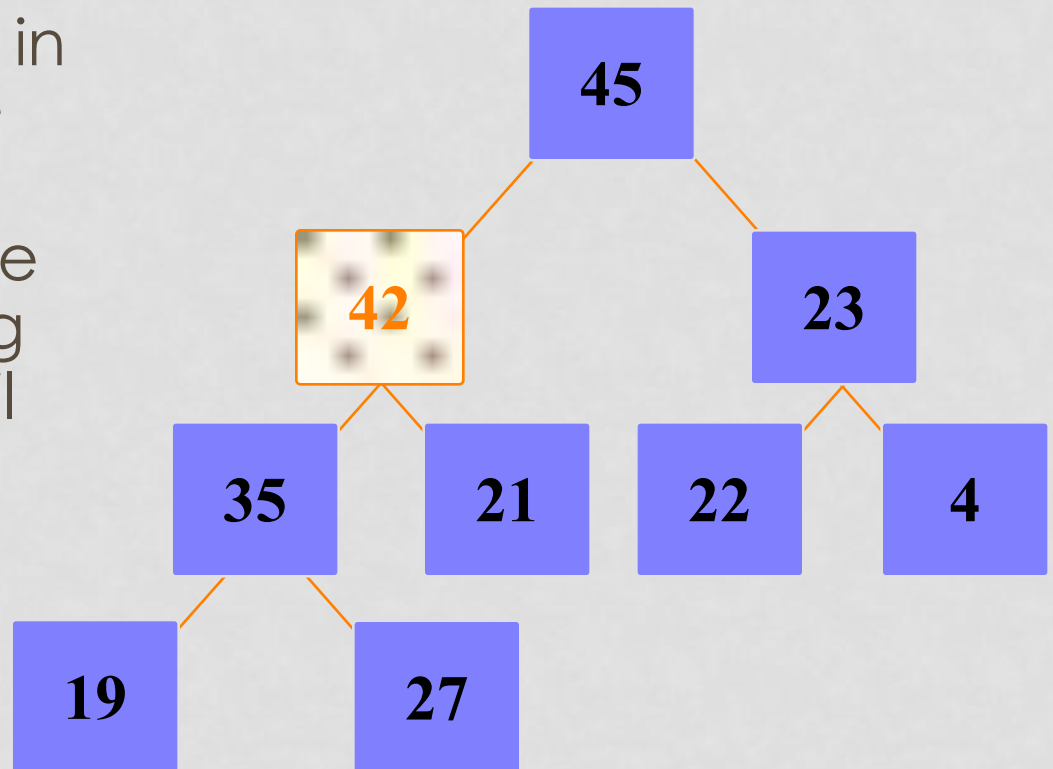
ADDING A NODE TO A HEAP

- ❑ Put the new node in the next available spot.
- ❑ Push the new node upward, swapping with its parent until the new node reaches an acceptable location.



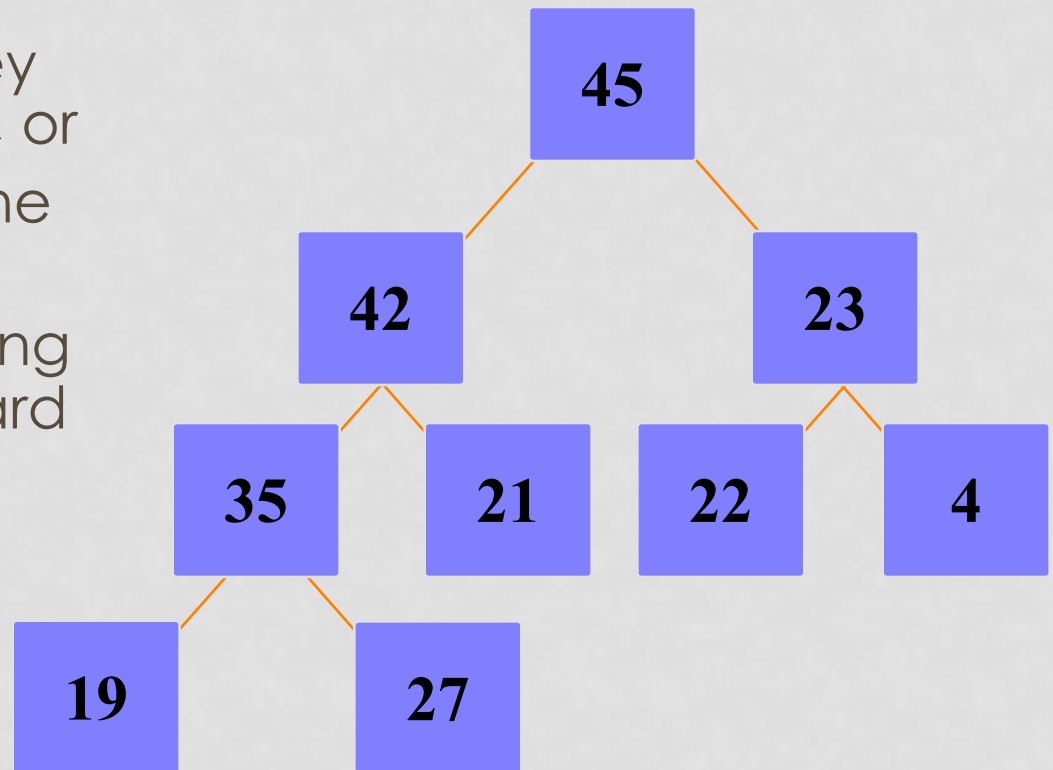
ADDING A NODE TO A HEAP

- ❑ Put the new node in the next available spot.
- ❑ Push the new node upward, swapping with its parent until the new node reaches an acceptable location.



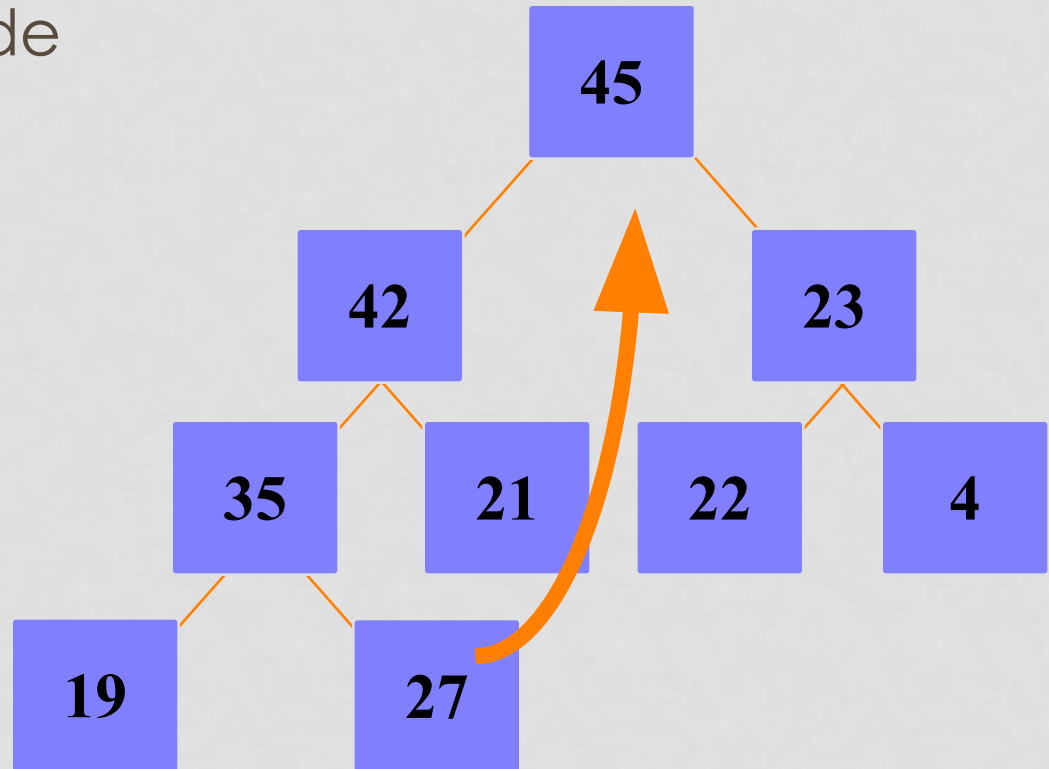
ADDING A NODE TO A HEAP

- ❑ The parent has a key that is $\geq$ new node, or
- ❑ The node reaches the root.
- ❑ The process of pushing the new node upward is called reheapification upward.



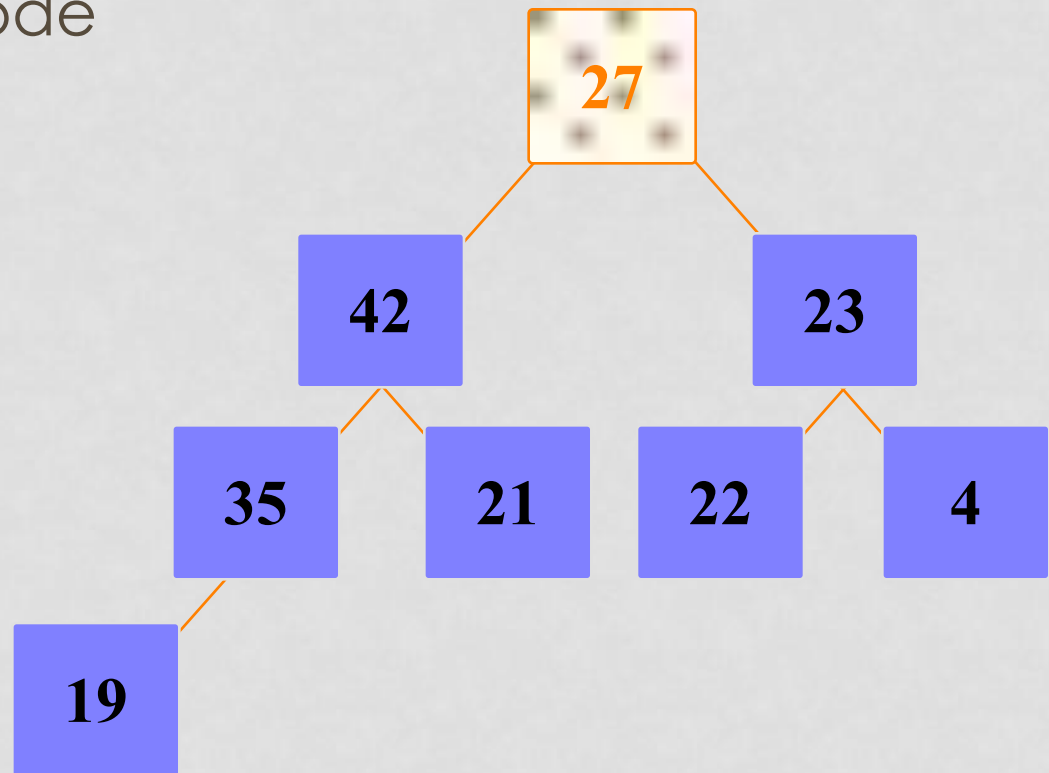
REMOVING THE TOP OF A HEAP

- Move the last node onto the root.



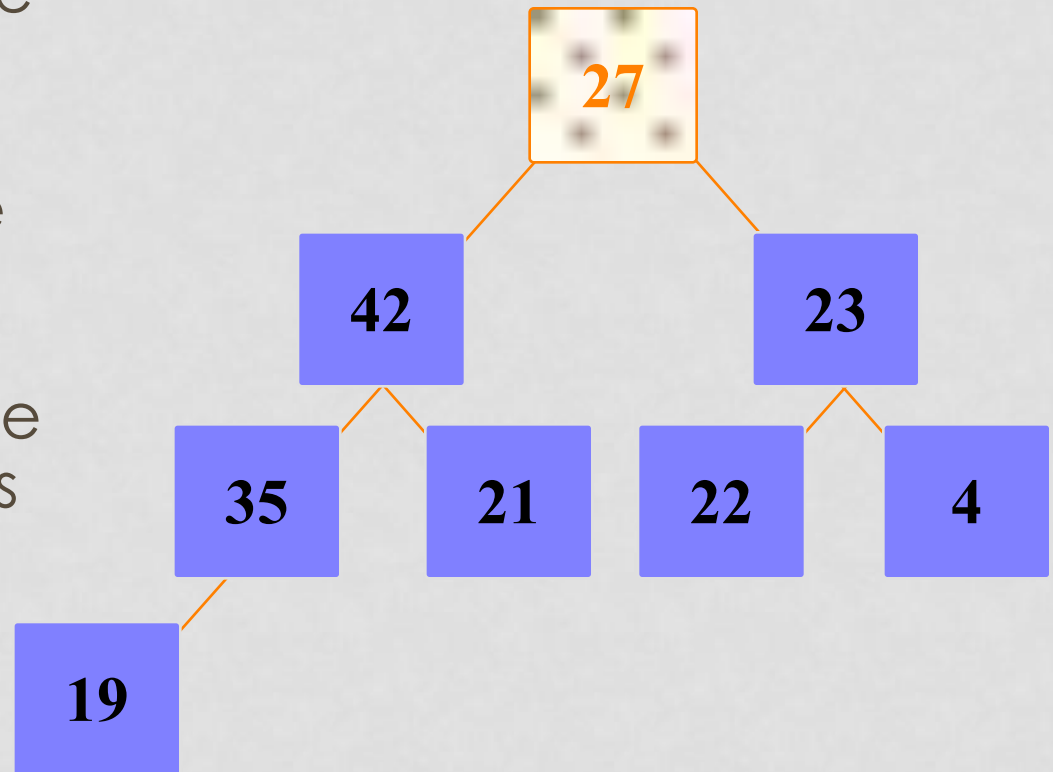
REMOVING THE TOP OF A HEAP

- Move the last node onto the root.



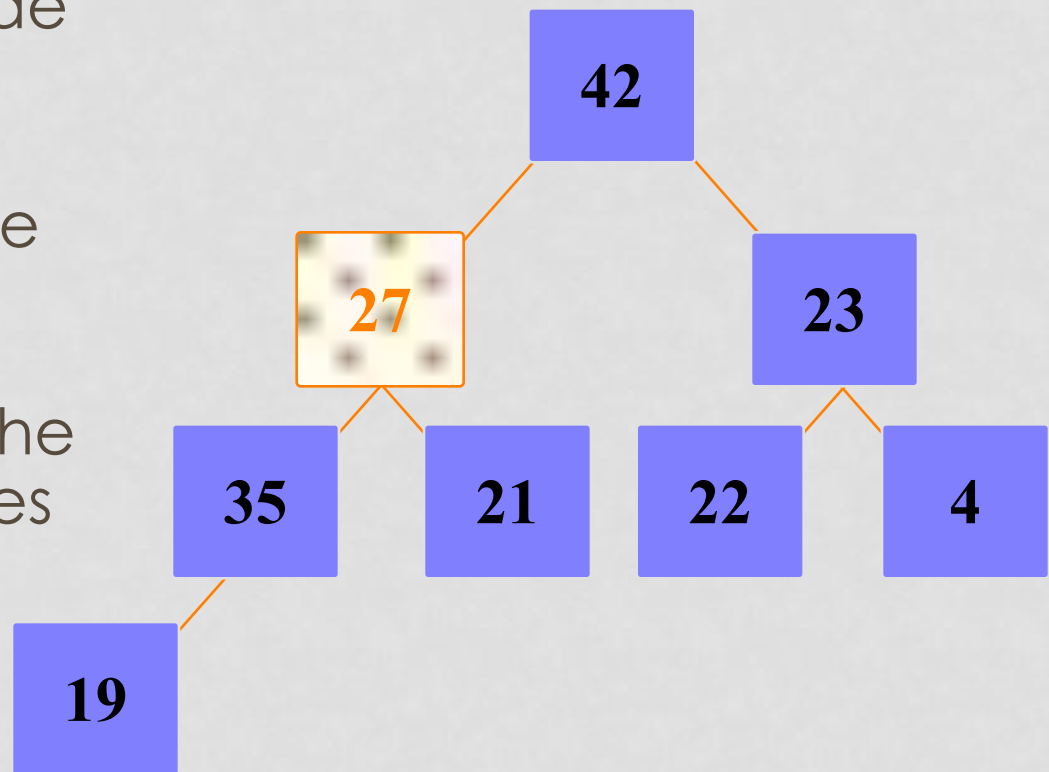
REMOVING THE TOP OF A HEAP

- ❑ Move the last node onto the root.
- ❑ Push the out-of-place node downward, swapping with its larger child until the new node reaches an acceptable location.



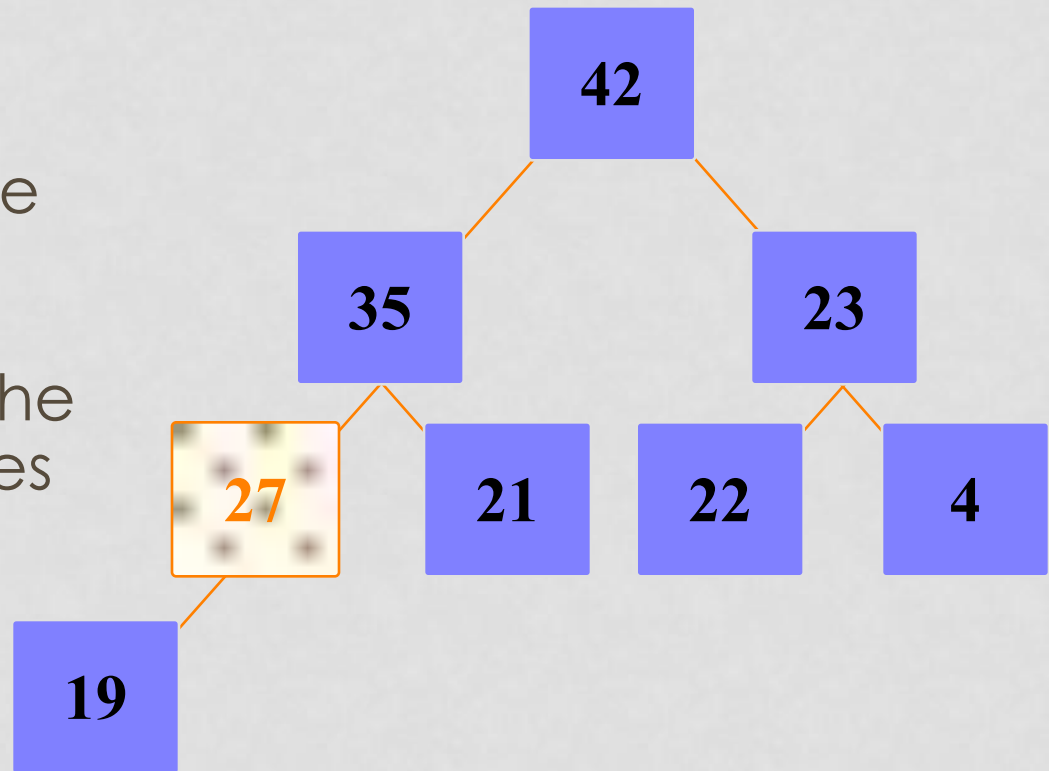
REMOVING THE TOP OF A HEAP

- ❑ Move the last node onto the root.
- ❑ Push the out-of-place node downward, swapping with its larger child until the new node reaches an acceptable location.



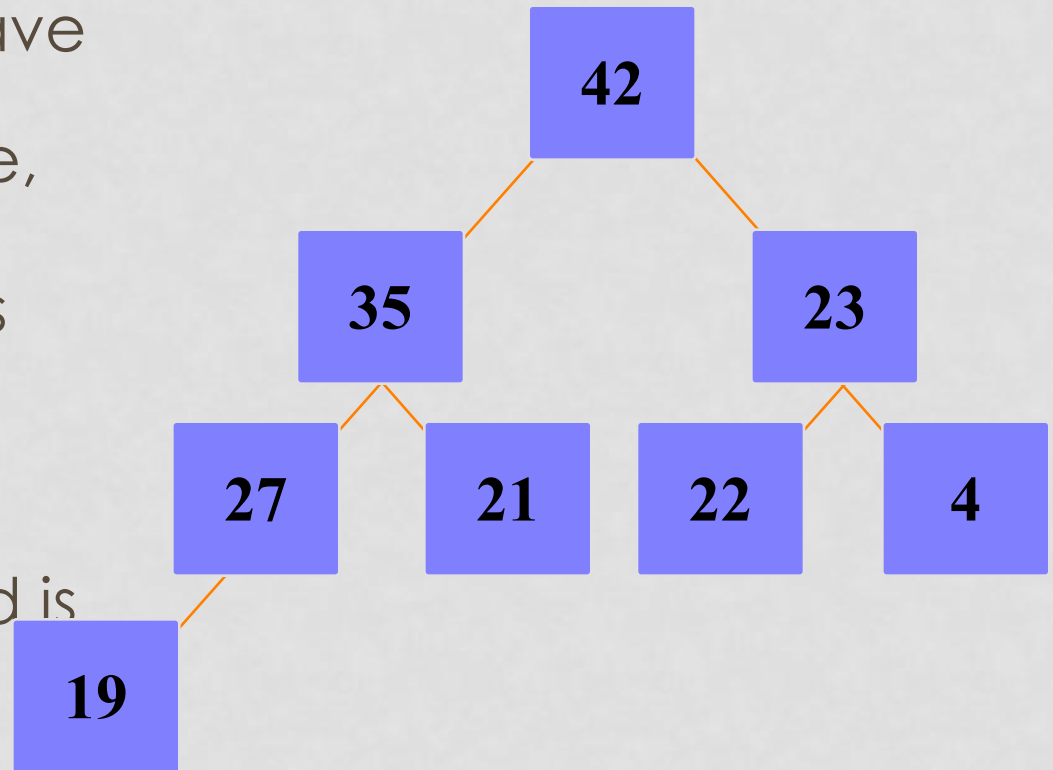
REMOVING THE TOP OF A HEAP

- ❑ Move the last node onto the root.
- ❑ Push the out-of-place node downward, swapping with its larger child until the new node reaches an acceptable location.



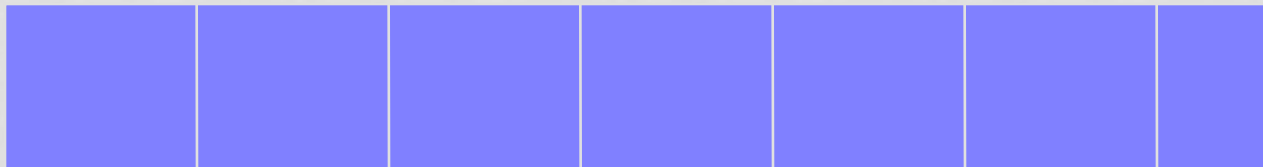
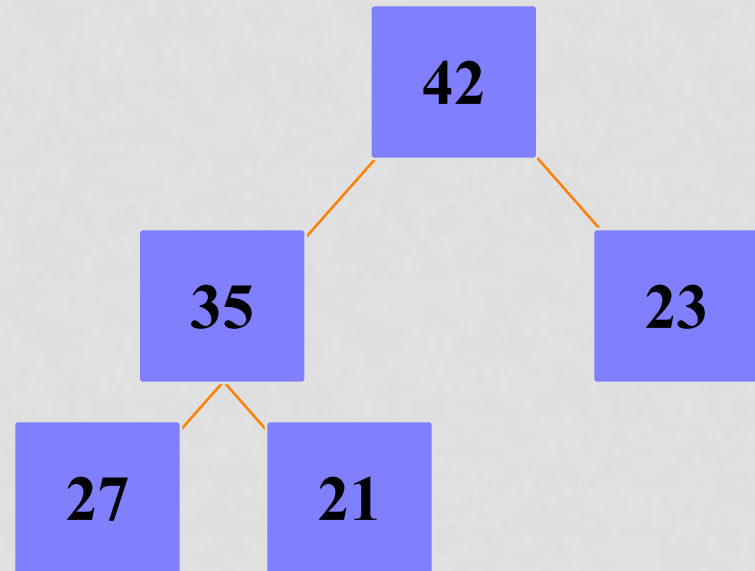
REMOVING THE TOP OF A HEAP

- ❑ The children all have keys $\leq$ the out-of-place node, or
- ❑ The node reaches the leaf.
- ❑ The process of pushing the new node downward is called reheapification downward.



IMPLEMENTING A HEAP

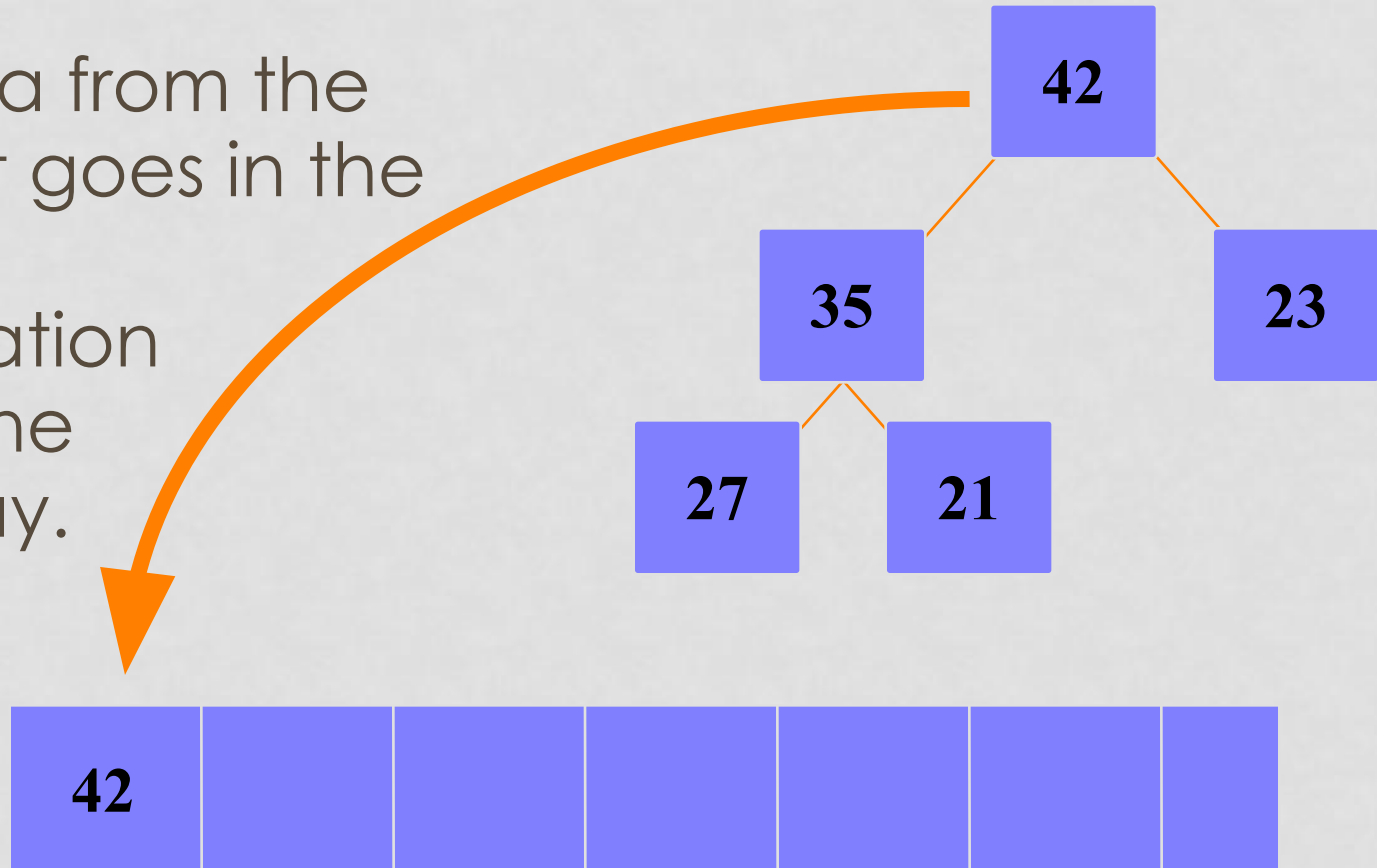
- We will store the data from the nodes in a partially-filled array.



An array of data

IMPLEMENTING A HEAP

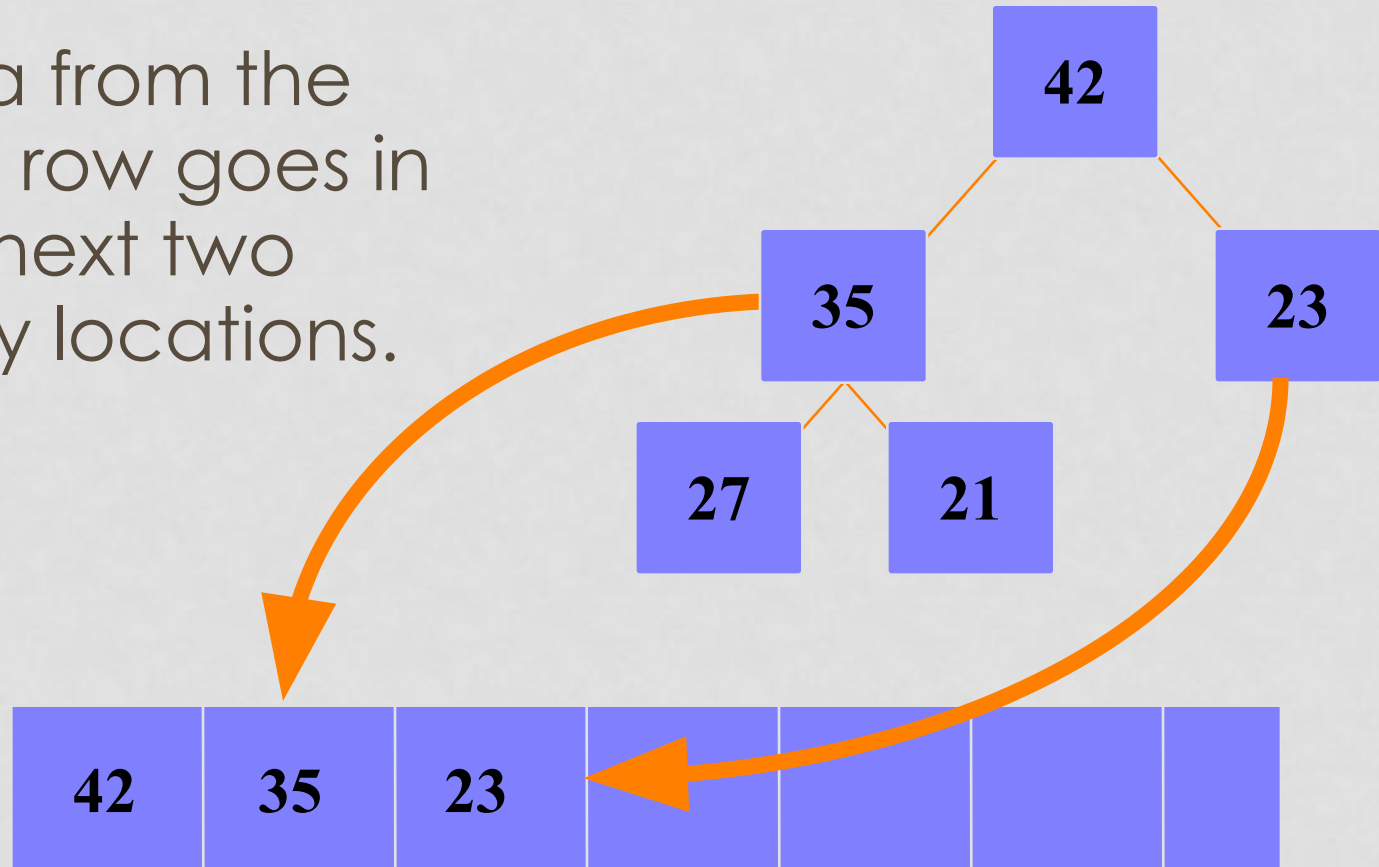
- Data from the root goes in the first location of the array.



An array of data

IMPLEMENTING A HEAP

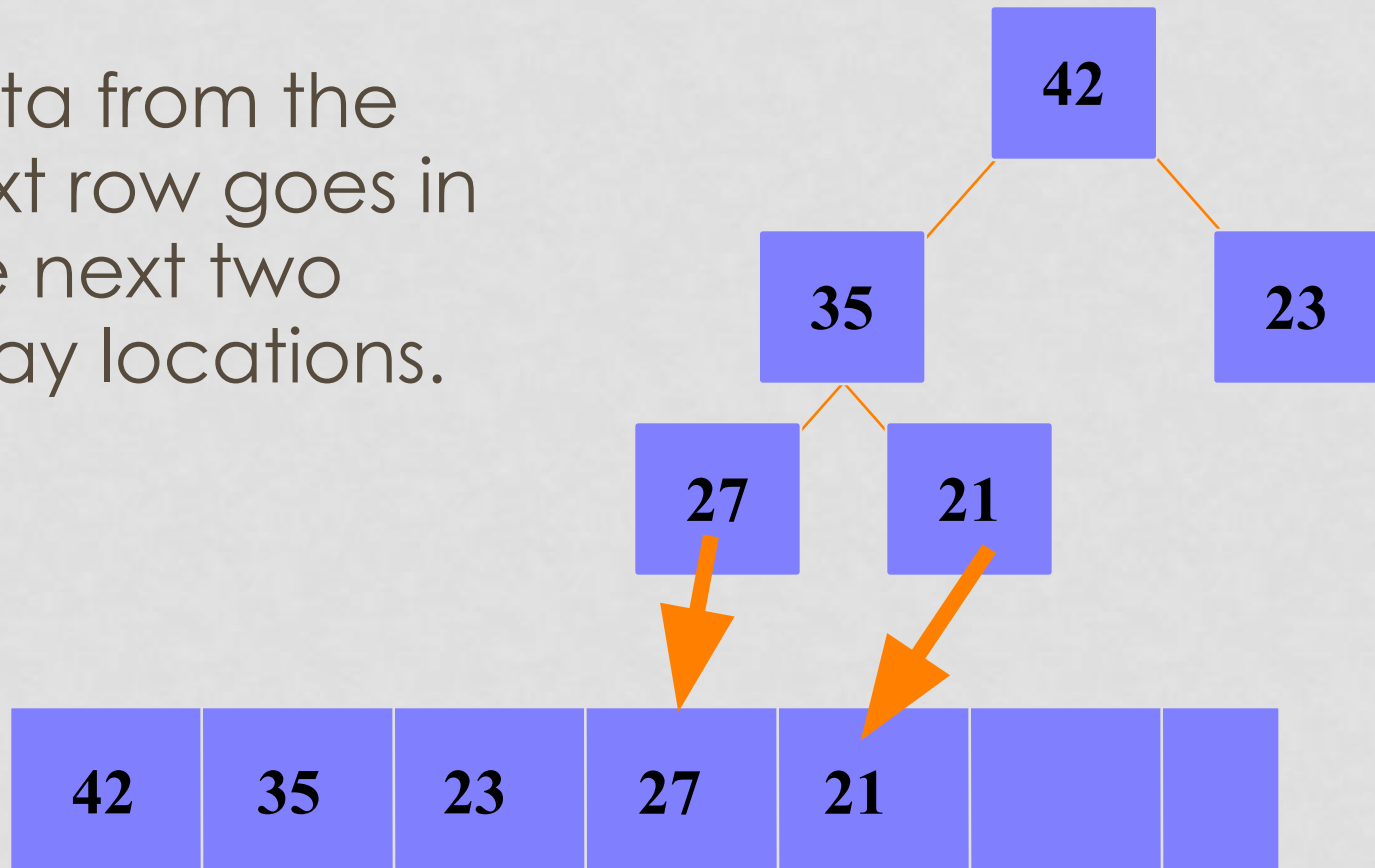
- Data from the next row goes in the next two array locations.



An array of data

IMPLEMENTING A HEAP

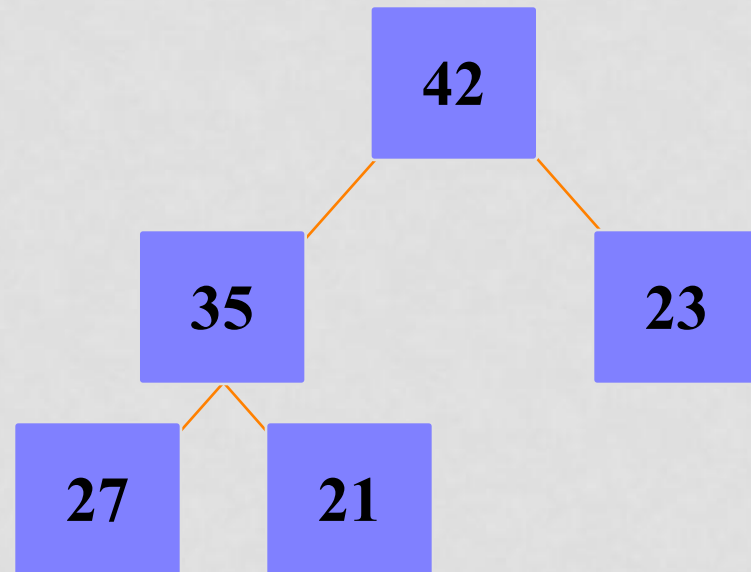
- Data from the next row goes in the next two array locations.



An array of data

IMPLEMENTING A HEAP

- Data from the next row goes in the next two array locations.

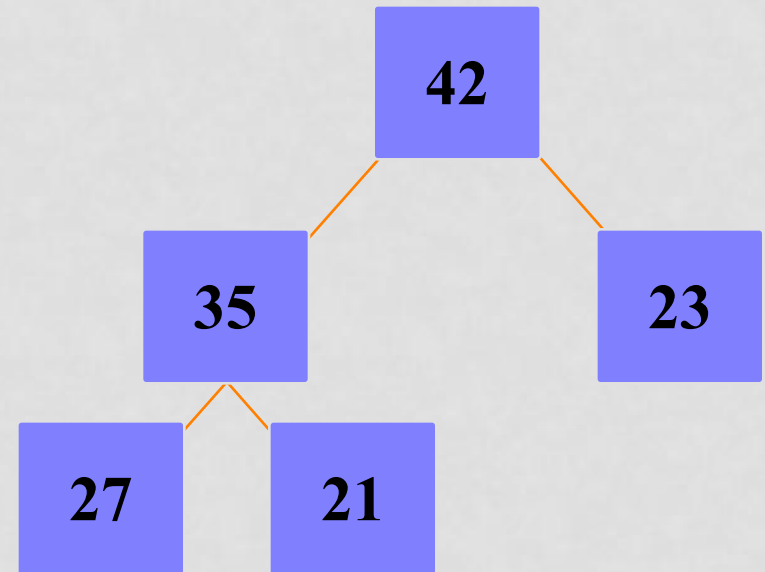


An array of data

We don't care what's in this part of the array.

IMPORTANT POINTS ABOUT THE IMPLEMENTATION

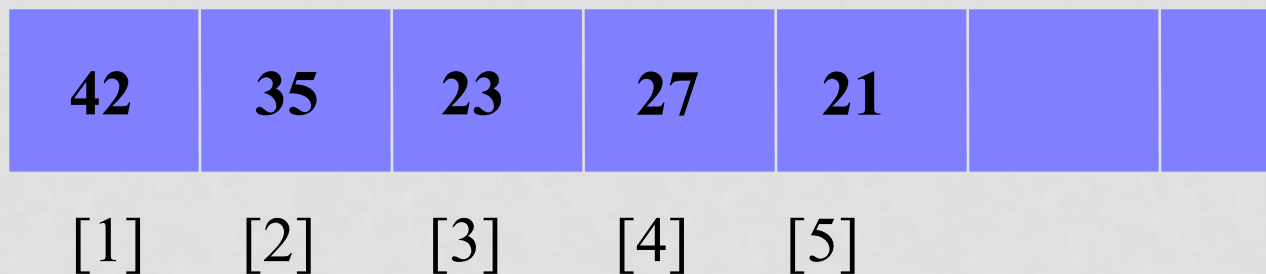
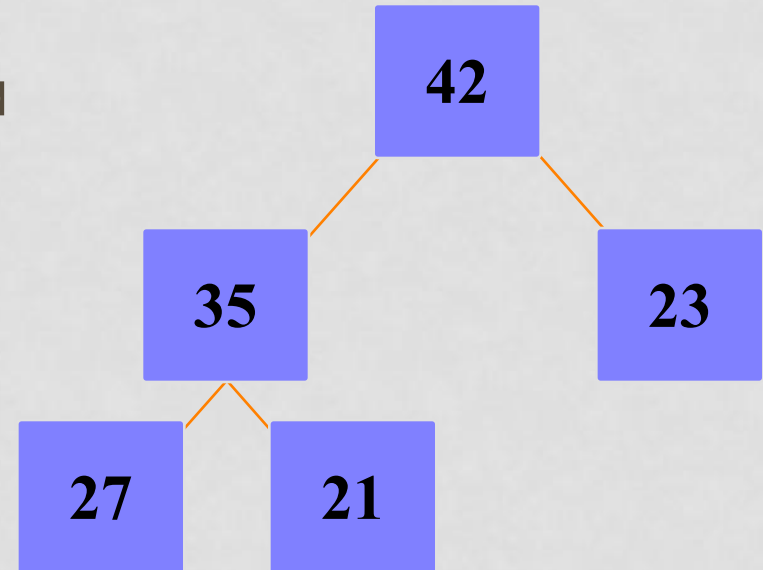
- The links between the tree's nodes are not actually stored as pointers, or in any other way.
- The only way we "know" that "the array is a tree" is from the way we manipulate the data.



An array of data

IMPORTANT POINTS ABOUT THE IMPLEMENTATION

- If you know the index of a node, then it is easy to figure out the indexes of that node's parent and children. Formulas are given in the book.



PRIORITY QUEUES

- **What is a priority queue?**
 - It is a queue with each element being associated with a "priority"
 - From the elements in the queue, the one with the highest priority is dequeued first

PRIORITY QUEUE SPECIFICATION

```
template<class ItemType>
class PQType {
public:
    PQType(int);
    ~PQType();
    void MakeEmpty();
    bool IsEmpty() const;
    bool IsFull() const;
    void Enqueue(ItemType);
    void Dequeue(ItemType&);
private:
    int numItems; // num of elements in the queue
    HeapType<ItemType> heap;
    int maxItems; // array size
};
```


PRIORITY QUEUE IMPLEMENTATION

```
template<class ItemType>
PQType<ItemType>::PQType(int max)
{
    maxItems = max;
    heap.elements = new ItemType[max];
    numItems = 0;
}
```

```
template<class ItemType>
PQType<ItemType>::MakeEmpty()
{
    numItems = 0;
}
```

```
template<class ItemType>
PQType<ItemType>::~~PQType()
{
    delete [] heap.elements;
}
```

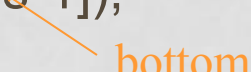
PRIORITY QUEUE IMPLEMENTATION

```
template<class ItemType>
void PQType<ItemType>::Dequeue(ItemType& item)
{
    item = heap.elements[0];
    heap.elements[0] = heap.elements[numItems-1];
    numItems--;
    heap.ReheapDown(0, numItems-1);
}
```



bottom

```
template<class ItemType>
void PQType<ItemType>::Enqueue(ItemType newItem)
{
    numItems++;
    heap.elements[numItems-1] = newItem;
    heap.ReheapUp(0, numItems-1);
}
```



bottom

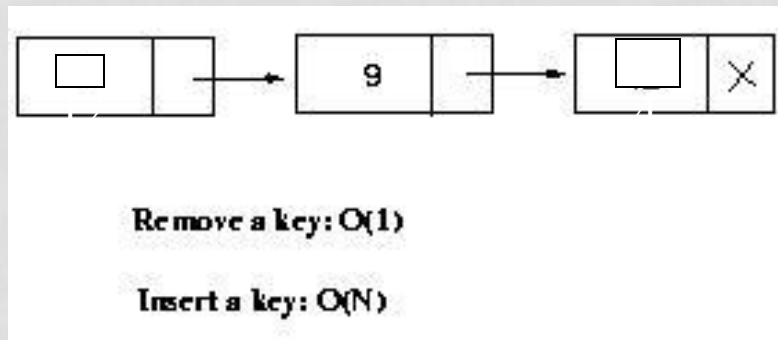
PRIORITY QUEUE IMPLEMENTATION

```
template<class ItemType>
bool PQType<ItemType>::IsFull() const
{
    return numItems == maxItems;
}
```

```
template<class ItemType>
bool PQType<ItemType>::IsEmpty() const
{
    return numItems == 0;
}
```

COMPARING HEAPS WITH OTHER PRIORITY QUEUE REPRESENTATIONS

- Priority queue using linked list



- Priority queue using heaps
 - Remove a key in $O(\log N)$ time
 - Insert a key in $O(\log N)$ time

CLASS TASK

- Create a heap and insert following values
12, 132, 14, 76, 44, 88

Delete all values. And note the sequence of deletion

132, 88, 76, 44, 14, 12

READING

- Chapter 11, Data Abstraction and Problem Solving using C++, Walls & Mirrors 3rd ed.